

Minimal Minos

Shree Krishna T.R.*

Kevin Sann*

thand008@umn.edu

sann0045@umn.edu

Abstract

In 2004, researchers proposed a control flow integrity protection system called Minos. Minos had extremely promising performance, but was never adopted for practical use because of the high implementation requirements. Namely, implementing Minos as the original researchers proposed would have required a 66 bit bus and the modification of memory hardware to add a separate custom DRAM chip called an "Integrity Bit Stuffer". The researchers themselves never actually accomplished this with a standalone machine, opting instead to simulate their design with the Bochs architectural simulator. Our contribution is an attempt to recreate their implementation, in the same simulator, only without modifying the memory architecture. If successful, then instead of using custom memory systems, this implementation would modify kernel behavior to store the relevant integrity bits in the existing memory hierarchy, for example by designating certain pages of memory to hold integrity data for the other pages in use. With this modification, the difficulty and complexity of implementation - at least in terms of hardware - goes down, but putting more workload on the kernel has its own drawbacks, and risks creating unacceptably high overhead for practical use.

ACM Reference Format:

Shree Krishna T.R. and Kevin Sann. 2024. Minimal Minos. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

CONTENTS

Abstract	1
Contents	1
1 Introduction	1
1.1 Our contribution	2
2 Related Work	2
2.1 Simulators	2
3 Background	2
3.1 Minos	2
4 Architectural Discrepancies	3
5 IA-32/Intel-64 Execution Model Discrepancies	3
5.1 The Pentium 4 Microarchitecture	3
6 The Linux Kernel	4
6.1 The Linear Address Space	5
6.2 The Slab	5
6.3 Memory Management Structs	6
6.4 Paging	6
7 Implementation Strategy	6
7.1 Architecture-Level Integrity	6
7.2 OS-Level Integrity	7
8 Limitations	7
8.1 age	7
8.2 simulation vs real life	7
8.3 OoO	7
9 Future Work	7
References	8

1 Introduction

The kernel is the core of an operating system. This core is responsible for handling privileged data, conducting privileged operations, and providing a level of abstraction from the machine for the user, among other things. In order to prevent damage or corruption to the kernel, computer systems often have an internal partition between "user space" and "kernel space", with kernel space being reserved and responsible for the more sensitive operations that a user or untrusted process should not directly handle (process scheduling and creation, access control, et cetera). This makes the kernel an obvious high value target for attackers. Corrupting or compromising the kernel effectively achieves the highest level of infiltration against a machine (ignoring things like virtual machine monitors, which create an additional, more privileged layer beneath the kernel but are not a part of a standard computer system). Taking control of the kernel gives the ability to control things like inter-process communication, many device

*Dept. of Computer Science & Engineering, University of Minnesota, 209 Vincent Hall, Minneapolis, Minnesota 55414. All authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2024 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . \$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

drivers which only run in kernel space, and networking components. Inserting malicious code into the kernel can allow it to persist and run invisible to user space (e.g. rootkits).

A very long-standing strategy for attempting to attack the kernel is return-oriented programming, wherein an attacker, through a memory vulnerability (e.g. overflow attacks), causes a program to jump to the wrong place after returning from functions [8]. By causing the program to land at a place of the attackers choosing, attackers can execute arbitrary code so long as that code is present on the victim machine. It doesn't actually need to be present as code, either; by violating byte alignment with the return address, the attacker can make the system "misread" data to mean different instructions than what was originally written into memory. If done well enough, this can give the attacker an improvised, Turing complete "language" with which to create an attack. If this is allowed to corrupt sensitive kernel data, then every aspect of the system is potentially compromised. This is the threat model Minos is designed to address.

Minos is a kernel space control flow integrity protection system that relies on marking data as either high or low integrity, using "integrity bits", and preventing low integrity data from being used in certain sensitive operations. Instructions are considered high integrity particularly with the Biba [1] integrity model in mind (which can be more explicitly referred to as no read down, no write up). This means that writing high integrity data cannot be done by instructions using low integrity data, and instructions handling high integrity data cannot access or view low integrity data as a part of its high integrity data handling. This strategy, if properly enforced, will theoretically prevent low integrity data from influencing or changing critical data for secure and correct control flow in the kernel. In order to enforce this, Minos tracks a single bit for each word of data set to either 0 for low integrity, or 1 for high integrity. This was implemented using two pieces of inconvenient custom memory hardware: a 66 bit bus for communicating words alongside their integrity bits, and a separate, custom "integrity bit stuffer" DRAM for buffering and handling integrity bits for data in active use.

1.1 Our contribution

In this work we seek to recreate minos twenty years later for the i686 Pentium 4 (i.e. "Willamette") [6]. Like the original Minos implementation, we will be implementing this version in the Bochs architectural simulator. In addition, we hope to cut down on the hardware modification requirements by integrating the functionality of the bus and IBS into the standard memory hierarchy (data caches and main memory). This will be achieved by modifying the linux kernel to store integrity bits separately but within the same layer of the memory hierarchy, e.g. separate pages store integrity bits for the non integrity bit pages.

2 Related Work

Minos is the primary mechanism we are basing our scheme off of [3]. Its security is assessed in [4]. It has been referenced primarily for its feature of hardware modification. By now, DIFT techniques have long been a part of the literature [3]. Methods of performing integrity checking besides hardware modifications involve software modifications such as pointer integrity checking, pointer authentication codes, and symbolic execution (e.g. the Elbrus E2K) [3]. Also, the IBM AS/400 had its own checks on pointer integrity, where pointers were tagged by the operating system and could only be manipulated by a limited set of instructions [3].

2.1 Simulators

To simulate, one may also choose such a microarchitectural simulator as Gem5. However, it is not feasible to perform testing fast enough to research this mechanism- it requires a heavy CPU load and operates at a level of abstraction lower than what we need it to. Given this fact, it is still necessary to use a lower-overhead simulator to conduct our experiments. Bochs simulating i686 in particular is good enough for our experiments because of the relatively low overhead to emulate an i686 machine, and the fact that

3 Background

3.1 Minos

Minos implements a 33rd bit for every 32-bit word in the physical address space to invariantly enforce a low watermark policy. To hold information for a 4096B page,

$$4096 = 2^{12} = 2^5 * 2^7 = 32 * 128 \text{ Bytes}$$

are required. The original Minos implementation is orthogonal to the memory model and is proven implementable on 64-bit x86 machines with flat address spaces [3]. This model was proven to work on any flat memory model system, and work with any paging put on top of it. It was tested on one processor (the Pentium 4 Prescott 336) with both Windows XP and two versions of Red Hat Linux (e.g. 6.2,9.0) [3]. It was claimed to include plenty of OS and architectural modifications. Upon contact of the original implementers, source code was given that provides evidence of architectural modifications made for Minos and gives hints as to how it interacts with the operating system.

All instructions and data in the Minos microarchitecture are augmented with integrity checking. This is done at the level of opcodes. Taking advantage of the fact that Bochs uses a `bxInstruction_c` class to represent each instruction, Minos tags each instruction so it may be thrown out accordingly should it not follow the integrity policy. Minos in addition tags each data accordingly to make sure that all writes are architecturally transparent. In-depth examination of the Minos source code provides plenty of insight into how the low-watermark policy was enforced [1].

DMA and Port IO are considered high integrity. This is especially apparent after examination of the APIC (Advanced Programmable Interrupt Controller) - the APIC is augmented

All General Purpose registers (e.g. EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI) are represented as `gen_reg_t` objects in Bochs augmented with an integrity value `reg_integrity`. When used during execution, the registers are changed accordingly at each Bochs sub-clock cycle (one run of `cpu_loop()` in Bochs is one clock) to track current results for current operations. It was not mentioned however, that segment register integrity is also considered to prevent segment accesses that break the low-watermark policy.

Operations for all kinds of data are completed using a full byte of integrity (e.g. either `0xffffffff` or `0x0`), irrespective of data size (e.g. byte, word, doubleword, quadword). This gives rise to integrity-checking, one byte at a time.

The low watermark policy of [1] is implemented via AND'ing two binary inputs together to force down or maintain integrity levels. Before execution, inputs are checked for integrity. After execution, outputs are checked as well.

The description leaves in many inferrable details, providing flexibility to their implementation. One may view this as room for flexibility from an ISA-independent standpoint, but their description raises many questions about their implementation - it does not reconcile its differences with the Pentium 4 microarchitecture or the microarchitecture modeled by Bochs.

4 Architectural Discrepancies

Although Minos executes on the Pentium 4, there remain several unexplained discrepancies between the OoO representation of Minos and the real OoO representation of the Pentium 4. It was found that the representations differed across the entire system. We compare the Minos description [3] to the Pentium 4 microarchitecture description [2, 5] as a baseline. We were surprised to find that these architecture-specific details were not discussed, as these details and their clarity are integral to accurate reimplementations.

Though Pentium 4 architecture has 32-bit and 64-bit variants, this does not matter - the implementation specifics will remain the same and is simply extended to the 3-level paging featured in 64-bit processors instead of 2-level.

There is no mention of PAE in the original Minos implementation. In our later discussion we provide this as grounds for future work, though we will not be covering use of the additional physical address bits to refer to integrity.

5 IA-32/Intel-64 Execution Model Discrepancies

In the IA-32/Intel-64 architectures, the CPL (Current Privilege Level) of the Processor is considered. Bochs gives credence to this fact, but these CPL's (e.g. not considered during execution in the Minos code. There is therefore no consideration for the current privilege level of the processor when performing integrity operations.

5.1 The Pentium 4 Microarchitecture

The Pentium 4 microarchitecture is composed of 4 central components: In-order Front End, Integer and Floating-Point

Execution Units, the Out-Of-Order Back End, and the Memory subsystem. We have provided two figures from around the era of the Pentium 4's market release to corroborate our understanding of Pentium 4 architecture, and pit this against another figure, detailing the amount of abstraction done on the part of Minos [3, 5].

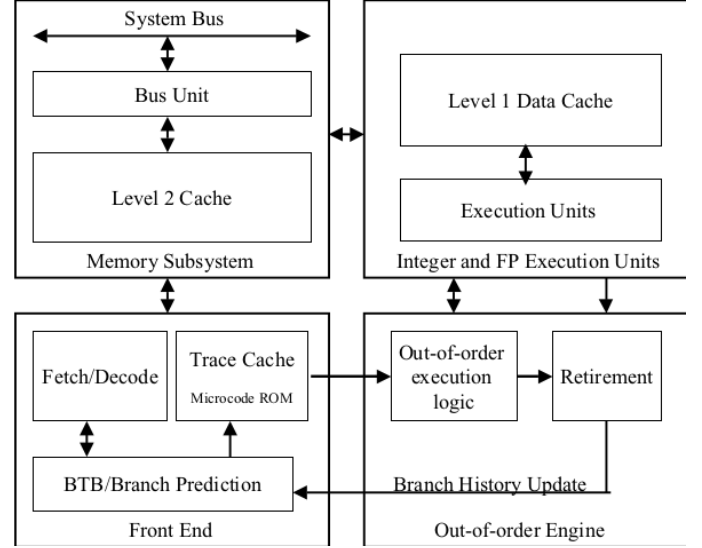


Figure 1: The Pentium 4 Microarchitecture

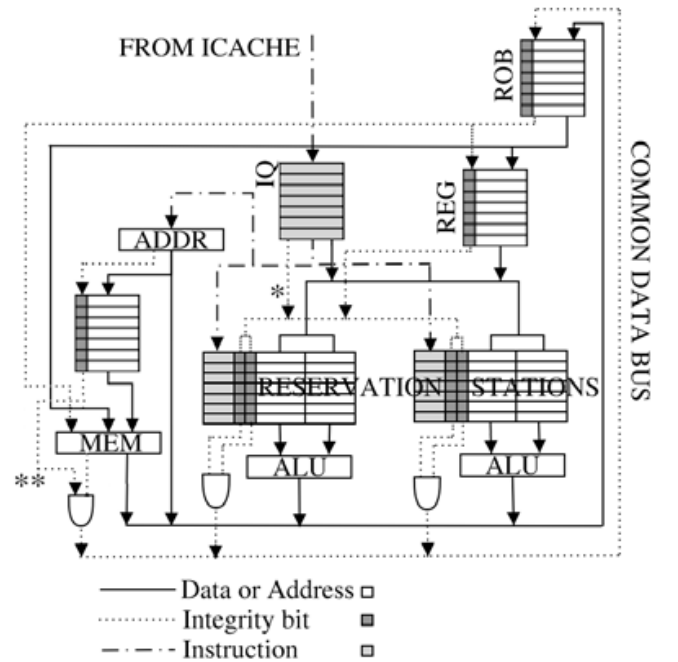


Figure 2: Minos on an OoO Processor

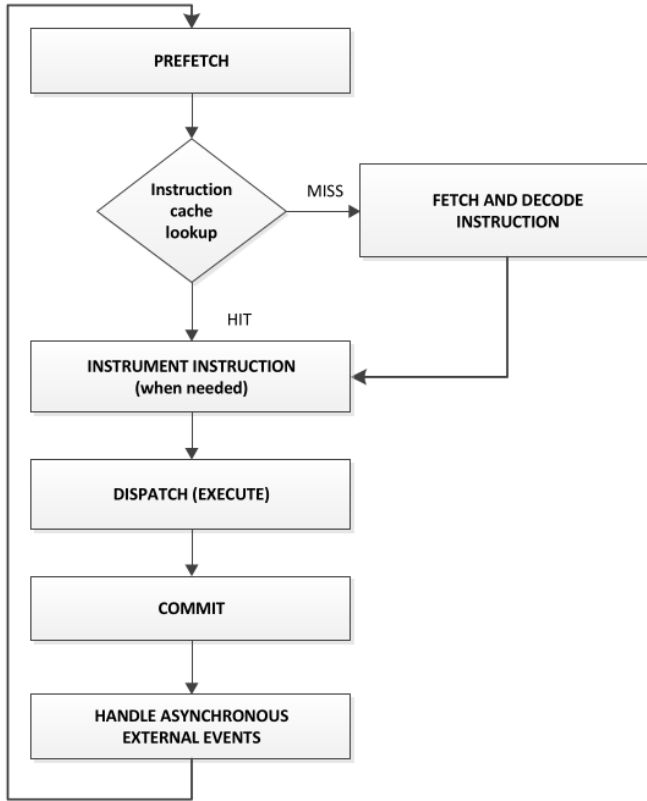


Figure 3: Execution of the Bochs `cpu_loop()`

5.1.1 In-Order Front End The Pentium 4 in-order front end consists of an ITLB, the front-end Branch Predictor (e.g. Front-End BTB), the IA-32 Instruction Decoder, the Trace Cache, and the Microcode ROM. For our discussion, we will primarily focus on all but the IA-32 Decoder and Microcode ROM - they are not points of introduction for low integrity.

BTB There is no obvious indication of a branch target buffer in Minos. However, there are obvious manipulations done on branch prediction calculations to perform integrity checking. Specifically, a call `i-jmodC0` is done to dereference integrity bits at respective arguments stored in either registers or memory for a jump address.

ITLB There is an ITLB that stores translations for instructions [5]. Its mention remains missing from the original analysis of Minos [3]. Looking at the source code, the ITLB is considered in such a way that integrity checking works with it. It interacts with the prefetch functionality embedded into the CPU, instead of manipulating a `bx_object` TLB class as would be done in Bochs 2.7.

Trace Cache .

In Minos, there was reference to both an "Instruction Cache" and a "Trace Cache". However, these are one-in-the-same inside the Pentium 4 and IA-32 literature [2, 5, 6]. Thus

their existence as separate entities is questionable for caching instructions. However, it was found there is an ITLB inside of Bochs 2.0.2, but implemented using a CPU prefetch function integrated into the CPU object, not a separate `bx_object` TLB class.

5.1.2 Integer and Floating-Point Execution Units The integer and floating-point execution units are where instructions are actually executed [5]. This section includes Integer/FPU Register Files, ALU's, the complex integer instruction units, the load and store address generation units, and the L1 data cache.

The Integer RF/Bypass network, FPU RF/Bypass network, and their interconnect to the L1 data cache are all abstracted to a RF, common data bus, and a reorder buffer. The CDB has been augmented as claimed in [3]. This leads to registers inheriting values from the CDB, discussed later.

MMX, BCD, SSE, SSE/2, and Floating Point units all fall under Floating Point and ignore input integrity. Evidence of this can be found in the FP register file. FP register file integrity is always forced to 0, and provides this value as auxiliary input in parallel with an operation.

The Integer execution units also follow a similar pattern in how their register file always forces integrity to 0.

5.1.3 Out-of-Order Execution Engine The function of the Out-of-Order Execution Engine in a Pentium 4 is to allocate, rename, and schedule. These units will help to prepare inputs properly at the respective execution units for true OoO execution. This is where the instructions are reordered and registers are renamed, being grouped and scheduled together so they fire immediately upon having ready operands [5]. These differences are not reconciled by Bochs or Minos - In-order execution is performed as there is no reordering of instructions to be done inside of the Bochs `cpu_loop()`.

5.1.4 Memory Subsystem The Memory subsystem is comprised of the L2 cache, off-chip DRAM, and the system bus. The L2 remains unmodeled by Bochs 2.0.2 and is therefore not covered by Minos, contrary to the claim that the code had such coverage. According to our findings, this remains invariant two decades later for Bochs - credence is only given to L2 in the small corner cases of (a) querying the CPUID and (b) the use of the CR0 register [6]. However, physical writes to DRAM truly become extended by a virtual Integrity Bit Stuffer module embedded within the code. The A20 is addressed in an orderly fashion, alternating between writes to the original intended address and another adjusted address for integrity bits to be stored in memory. Writing the value of the attributed integrity to the additional two bits by which [3] extends the System bus to 66 bits. This is while the system bus is originally 64 bits in the Pentium 4 microarchitecture.

6 The Linux Kernel

In order to implement Minos without adding custom hardware support, the linux kernel needs to handle the integrity bit logic, from assignment to storage to passing the appropriate bits alongside data as it propagates back and forth

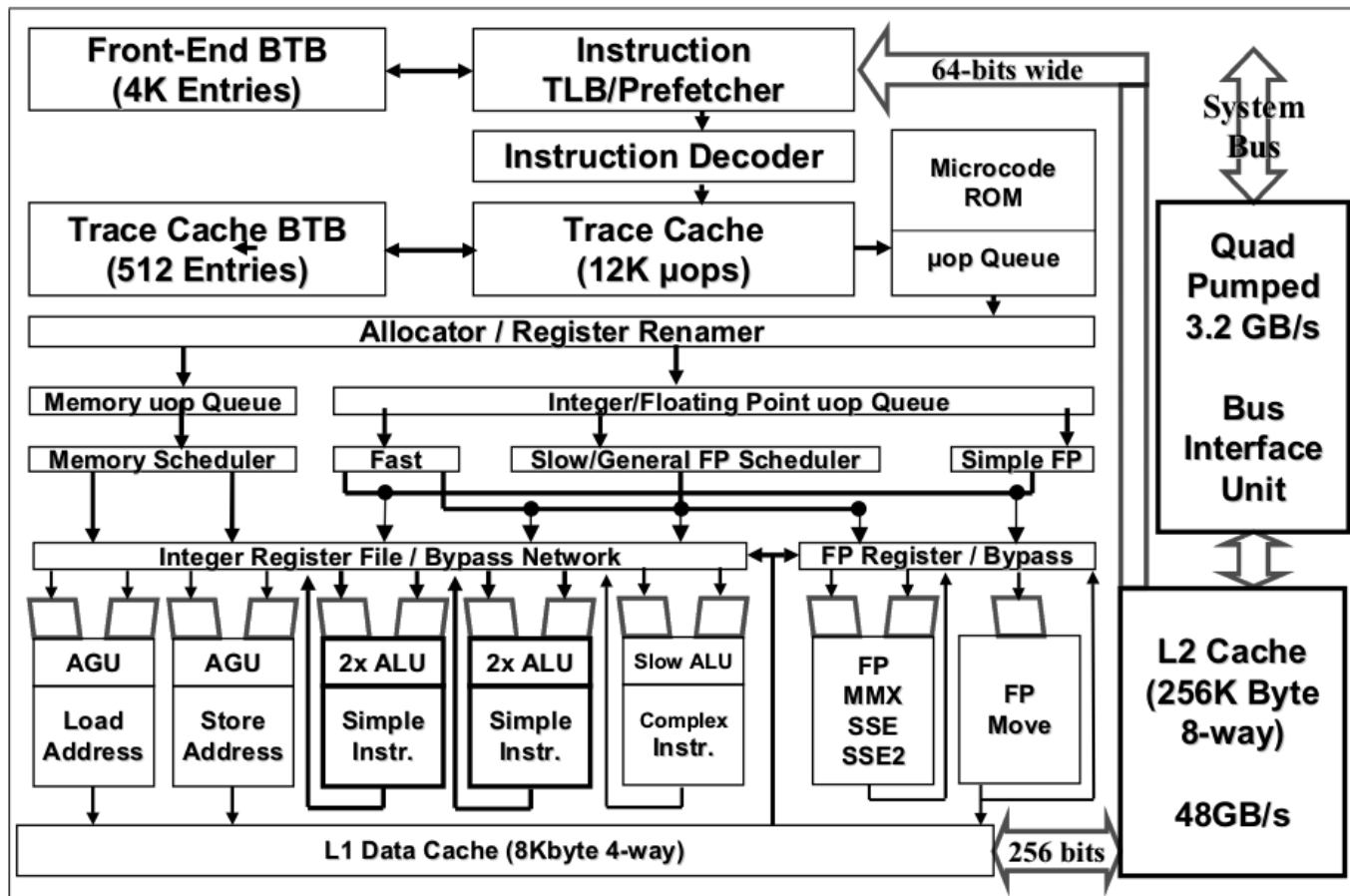


Figure 4: An IA-32/Intel-64 Segment Descriptor.

between main memory, caches, and use in instructions. This section covers some relevant components of the kernel for the implementation of Minos' integrity bits.

6.1 The Linear Address Space

The simulated system uses a flat, 4 gigabyte memory model, where memory can essentially be thought of as a 4 gigabyte array of untyped words. Within this model, segmentation is enabled; this allows large segments of memory to be effectively partitioned from each other (to various ends). In order to mark these borders, there exists segment descriptors in the kernel. There are several types of segment descriptors (e.g. Data Segment DS, Stack Segment SS, Global Segment GS, Extra Segment ES, File Segment FS).

But they all use the same format, as per Figure 5.

Surprisingly, 64 bit segment descriptors are still used in the 32 bit architecture; segment descriptors are a struct containing the two unsigned long variables a and b, where a is the second row of the figure, and b the first. Of particular interest is the AVL bit; this bit is specifically set aside for use by system software [6].

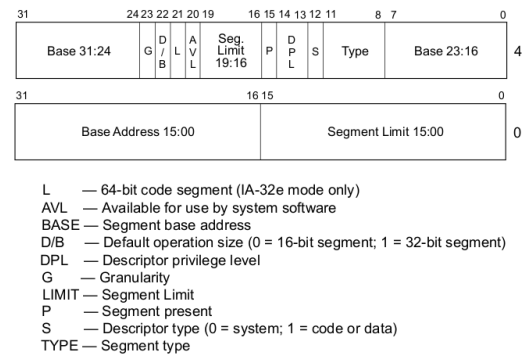


Figure 5: An IA-32/Intel-64 Segment Descriptor.

6.2 The Slab

The Linux slab is dedicated to storing frequently used Kernel data structures. These may reserve positions inside caches to do so [9]. This is specifically pertinent to `mm_struct`. If one performs a `cat /proc/slabinfo`, s/he will find it is one of

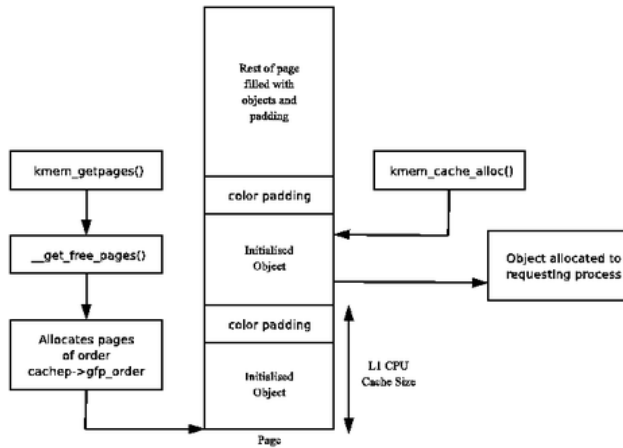


Figure 8.2: Slab page containing Objects Aligned to L1 CPU Cache

Figure 6: The Slab, aligned to an L1 Cache

the topmost entries returned, and may examine how many objects it has. Minos adds 128 Bytes to the slab allocator. The only way this is foreseeably done is via adding these bytes to the `mm_struct` and tracking, by 128 bytes, only enough integrity information to represent a page of each process. Later, we discuss how they presumably trampoline off of this idea in order to perform virtual memory swapping with the swap cache (also located in the slab).

6.3 Memory Management Structs

6.3.1 `mm_struct` `mm_struct` is a kernel data structure that describes the process address space. On a real machine, it can be stashed in the cache by way of consistently reserving space for it there [9]. It is per-task/process.

6.3.2 `vm_area_struct` The `vm_area_struct` describes a specific section of contiguous virtual memory space [9]. This is specific to processes and is what is used by the kernel to track process memory mappings. One may view a full list of mapped regions for a process via `cat /proc/PID/maps`, where PID should be replaced with the desired process id. With such being the case, the `vm_area_struct` has an `mm_struct` pointer within its fields.

6.4 Paging

In order to efficiently use physical memory space in a system with multiple processes and dynamic programs constantly vying for active (“live”) memory space, all modern operating systems implement paging. Physical memory is divided into evenly sized pages (4096 bytes, or 4 kilobytes, is more or less standard), and processes place their data into said pages when they are active and the process in question has access to a given page. As physical memory runs out of space, the kernel selects pages to replace in physical memory using a predefined algorithm (page management). The pages to be swapped out are stored in persistent memory (i.e. on disk) and their copies in physical memory are replaced with pages

that hold newer data. As the old data becomes needed again, the page management system swaps them with other pages currently in physical memory pages so that they can be used again. This allows a much greater amount of memory than is actually available in main memory to be active at once.

In order to keep track of what data on disk could potentially be called into memory, pages on disk are stored in a designated section called the swap cache. This section holds the pages in page format until the process or processes that were using them all exit; after this, data in the swap cache is freed [9].

`mem_map_t`

The `mem_map_t` data structure (also known as page) is used to hold information about each page of physical memory.

```
typedef struct page {
    /* these must be first (free area handling) */
    struct page *next;
    struct page *prev;
    struct inode *inode;
    unsigned long offset;
    struct page *next_hash;
    struct page *count;
    unsigned flags; /* atomic flags, some possibly
                     * updated asynchronously */
    unsigned dirty:16;
    unsigned age:8;
    struct wait_queue *wait;
    struct page *prev_hash;
    struct buffer_head *buffers;
    unsigned long swap_unlock_entry;
    unsigned long map_nr; /* page->map_nr == page - mem_map */
} mem_map_t;
```

Figure 7: A struct page

In order to keep track of page metadata, the kernel has a struct called page (e.g. `mem_map_t`) which holds assorted relevant data about each memory page. This includes 4 bytes of flags, a struct indicating the least recently used page, and age and use counters, among other things. Many items in this struct are lock protected.

7 Implementation Strategy

Keeping step with the original Minos work, our system is simulated with Bochs. In a related project, the simulated processor was modified to be Minos compatible- an unavoidable action unless the decision had been made to make this Minos only implementable as a fully virtualized system. The kernel is then to be modified in several places.

7.1 Architecture-Level Integrity

First, integrity data needs to be included, both in new structures created for this project, and existing structures responsible for memory management and maintenance. We have located where Minos has accounted for integrity – this is very much the case for the TLB. In doing so, we have devised our own scheme based on Minos to combat control flow data overwrites, specifically based on the TLB’s. We augment TLB entries using an extra number of bits. We have a 1-bit flag to check for whether the tlb entry references an integrity page or not. This is provided with an 8-bit offset into an integrity page. In the case that it is a normal process and the offset for the corresponding DTLB integrity entry. In Bochs 2.7, there are both ITLB’s and DTLB’s. We can treat entries of the two differently based on if they are translations of integrity

pages.. If the entry refers to an integrity page, it is skipped until the time of use. If the entry doesn't refer to an integrity page (e.g. `!is_integrity`), then the entry's integrity within reserved cache space .

7.2 OS-Level Integrity

In order to maintain integrity bits on a virtual page level, we propose the following schema. In order to maintain the integrity bits for each page, certain pages will be marked as integrity pages. There will be at least one integrity page per process, and this page will be resistant to swapping except during context switches in which all pages it tracks are also swapped. This resistance comes in part from the fact that each page it tracks will repeatedly access it to collect integrity information. This will generate significant overhead, as now each memory access is two accesses. To try and reduce this overhead, the non integrity pages will be split into two types, as follows. Each page has an associated struct of type "page", as discussed before. Our implementation adds two variables to the struct: an unsigned char named `is_integrity` and a pointer to another page struct named `ptr_integrity`. The `is_integrity` char serves to identify the type of page, in increasing level of integrity, and can have a bitwise value of 0,1,or 2. To remember which is which, we use the following analogy: there are generals, soldiers, and civilians. 0 indicates that this page is a "civilian" and contains no integrity data (i.e. a page holding entirely non-control-sensitive data). The `ptr_integrity` value here is set to null, and no second memory check is needed when accessing this data: everything here is low integrity, so there is no need to waste time asking if something should be trusted. 1 indicates that this page is a "soldier" and may contain some integrity data. The `ptr_integrity` value here is set to the relevant integrity bit storing page (the one holding this page's integrity bits). The soldier might have the authority to hold and provide high integrity data, but it needs to check with the general (the integrity page) for permission to use any data it holds as high integrity data. 2, of course, indicates that this is an integrity page. `ptr_integrity` is null here, just like with civilians, as integrity bits must be considered high integrity to be trusted as arbiters of integrity themselves, so there is no higher authority for that pointer to point to. In order to maintain integrity on a cache level, a different strategy must be implemented. Caches operate at a dramatically smaller granularity data-wise, and don't have as much space to leave to integrity bits. Our strategy here is to create a small subsection of the cache, and attempt to sustain the presence of integrity bits in that line. Since caches, unlike virtual memory, have a very explicit and static size, the exact number of integrity bits needed to maintain it are constant and known. Therefore, the Minos implementation here is to insert integrity bits into an appropriate number of cache lines and use them as a bitmap; each integrity bit, based on its position in the integrity lines, matches with a given section of a given line in the cache (if lines only hold one word, as is the case with some L1 cache designs, then this would mean

one bit per line). This means two cache accesses occur for each data access in the cache; one to grab the appropriate cache line, then one to grab the appropriate byte to read the relevant bit(s) of integrity for the parts of the data line being read. In order to update the cache, two writes would be needed: one to modify the data line, and one to modify the appropriate integrity line and reinsert it into the cache.

8 Limitations

8.1 age

In basing our implementation on similar technology to what the original Minos used, our work uses seriously outdated technology; from simulating a nearly 20 year old processor to using an over 20 year old architectural simulator and version of linux, effectively every individual component used in the implementation could be updated. Each of these updates, of course, would have potentially major implications for the implementation of Minos. Some would be less concerning: for example, linux likes to minimize how much they change and deprecate with updates, electing instead to build on and extend existing designs, which could mean minimal change. However, some other updates, like modernizing the processor, could have major implications, due to the growth of branch prediction and highly speculative pipelines.

8.2 simulation vs real life

As described above, the simulated processor was modified for Minos compatibility. While this would be dramatically less serious a change than the original Minos, it would still require support from a cpu manufacturer or custom production to create in the real world. Moreover, the discrepancy in cache structure means even if we made a complete evaluation of system performance in terms of speed penalty caused by Minos, that evaluation would not guarantee any scaling with a real life Minos system.

8.3 OoO

While out-of-order execution has become standard in modern processors, and exists in the real version of the processor we simulated, Bochs does not actually have true out-of-order execution capability. Bochs itself does not reorder instructions in order to perform execution. Instructions are executed in order and aren't rearranged the way in which a real Pentium 4 would. Clearly this is a consequence of the fact that Bochs was based on an in-order processor execution model [7]. As a result, this simulation fails to represent the true range of edge cases an out-of-order processor might display, and therefore fails to represent and test this near universal feature of modern commercial processors with Minos in place.

9 Future Work

First and foremost among the interesting extensions to this project would be the updating and testing of this implementation strategy to operate with a modern processor's speculation practices. This would require another future project:

switching the architectural simulator to a more modern alternative. GEM5 has been suggested, as has VirtualBox. This would require a significant refactor to accommodate the changes in how the architecture is represented between the Bochs simulator and its more modern contemporaries. However, this would also give an opportunity to create a more true-to-reality implementation than the Bochs version was capable of, due to the architectural discrepancies described earlier. Additionally, it was suggested that for the purpose of implementing cache storage for integrity bits, a strategy called cache coloring could be used. Cache coloring takes advantage of the fact that many cache management policies select cache lines for data in a deterministic or semi-deterministic manner. Instead of repeatedly inserting the integrity cache lines to keep them in place, cache coloring would involve managing addresses of data and instructions to prevent them from ever being assigned to the cache lines with integrity bit data. This would require some binary rewriting to ensure that compiled kernel structures don't get addresses that would resolve to the integrity partition within the cache, as well as some more complicated address use; either that, or somehow the cache replacement logic would need to be modified. The operating system used here, Red Hat Linux 9.0, was very old. This had advantages; it was smaller and simpler than the modern linux equivalent, and was smaller and easier to run on Bochs, which heavily limits the resources that can be allocated to a simulation to the point that it is impossible to simulate many more modern operating systems. However, newer operating systems, as well as ones derived from different sources, such as a windows version instead of another linux, could be interesting to try implementing this system with as well (not to mention more representative of how challenging it would be to make this standard in the modern day). To do this, like with the more modern processors extension possibility, a new architectural simulation would have to be selected.

References

- [1] K. J. Biba. 1977. Integrity Considerations for Secure Computer Systems. *MITRE Technical Report* (1977).
- [2] Doug Carmean. 2000. *Inside the High-Performance Intel Pentium 4 Processor Microarchitecture*. Retrieved Apr 13, 2024 from <https://courses.cs.washington.edu/courses/cse548/05wi/files/Pentium4-Microarchitecture-Slides.pdf>
- [3] Jedidiah R. Crandall and Frederic T. Chong. 2004. Minos: Control Data Attack Prevention Orthogonal to Memory Model. In *Proceedings of the 37th Annual IEEE/ACM International Symposium on Microarchitecture* (Portland, Oregon) (*MICRO 37*). IEEE Computer Society, USA, 221–232. <https://doi.org/10.1109/MICRO.2004.26>
- [4] Jedidiah R. Crandall and Frederic T. Chong. 2005. A security assessment of the minos architecture. *SIGARCH Comput. Archit. News* 33, 1 (mar 2005), 48–57. <https://doi.org/10.1145/1055626.1055634>
- [5] et. al. Glenn Hinton, Dave Sager. 2001. The Microarchitecture of the Pentium 4 Processor. *Intel Technology Journal Q1* (2001).
- [6] IA-32 Basic Architecture 2024. *Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 1: Basic Architecture*. Retrieved Apr 13, 2024 from <https://cdrdv2.intel.com/v1/dl/getContent/671436>
- [7] Stanislav Schwartzman and Darek Mihoka. 2012. *How Bochs Works Under the Hood 2nd Edition*.
- [8] Hovav Shacham. 2007. The geometry of innocent flesh on the bone: return-into-libc without function calls (on the x86). In *Proceedings of the 14th ACM Conference on Computer and Communications Security* (Alexandria, Virginia, USA) (*CCS '07*). Association for Computing Machinery, New York, NY, USA, 552–561. <https://doi.org/10.1145/1315245.1315313>
- [9] Understanding The Linux Virtual Memory Manager 2007. *Understanding The Linux Virtual Memory Manager*.